Parameter-Efficient Fine-Tuning (PEFT)

Michael Galarnyk

ODSC AI West · October 29, 2026

THE PROBLEM

Full fine-tuning a 4B model needs ~38 GB of GPU memory.

THE IDEA

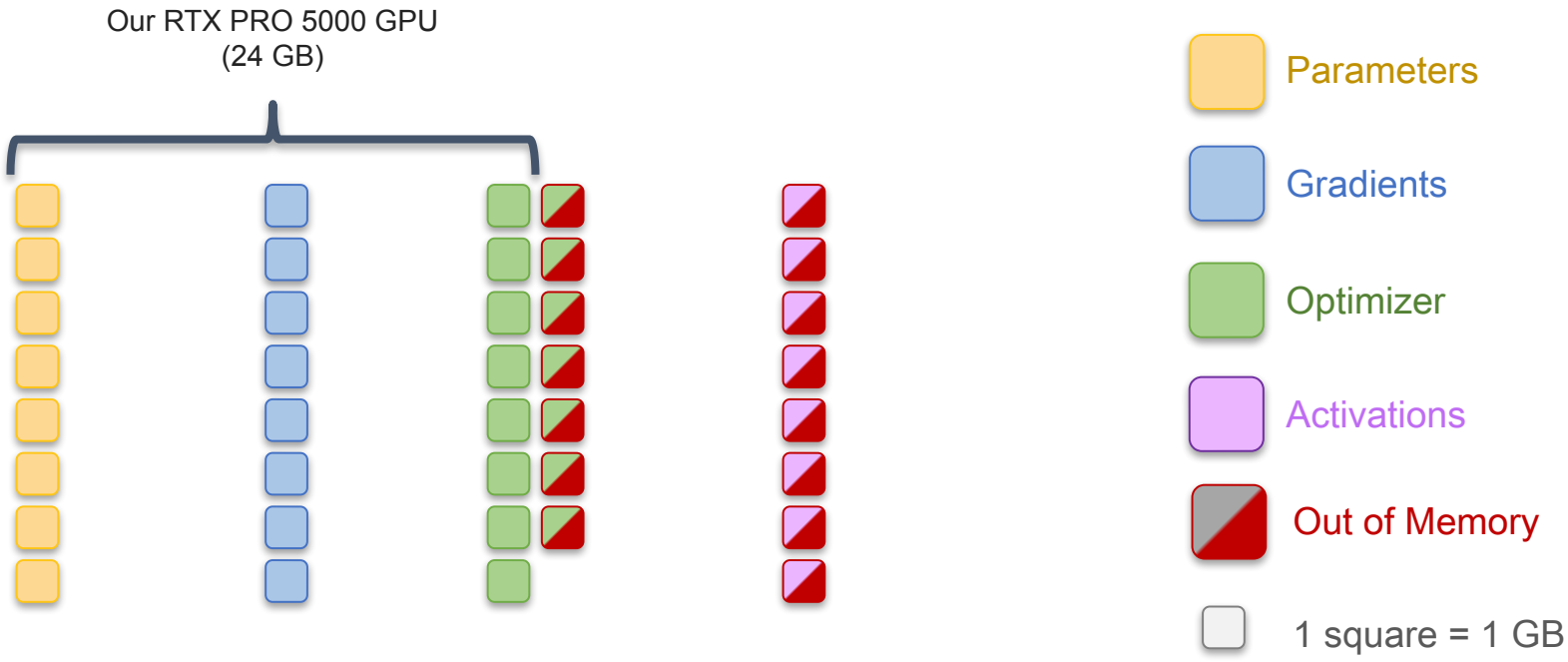
Freeze the base model. Train a small set of new parameters.

THE RESULT

Fine-tune large models on one GPU with matched task quality.

Full fine-tuning is memory-bound

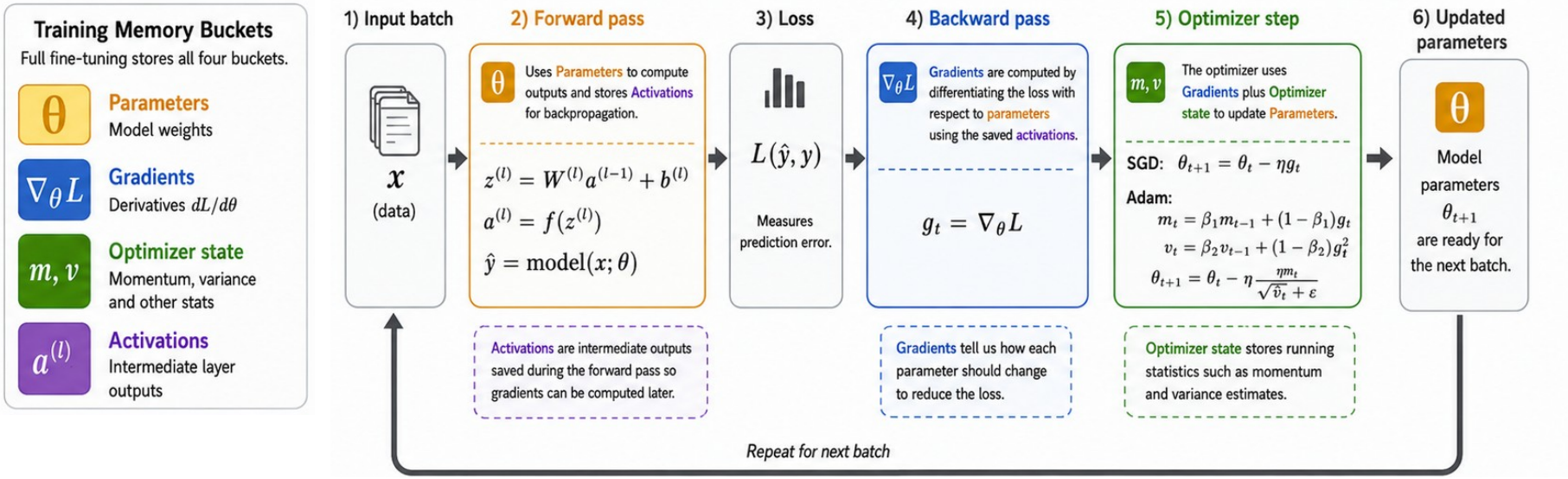
Full fine-tuning Nemotron 3 Nano 4B: gradients + optimizer dominate the ~38 GB



Parameters	Gradients	Optimizer	Activations	Total
(bf16) 7.5GB	(bf16) 7.5GB	(bf16) 15GB	(bf16) 7.6GB	~38GB needed

Training memory buckets across the forward and backward pass

Each bucket has a role in the training loop; gradients and optimizer state exist only because we are training.



Background

Why fine-tune a base LLM

Adapt one base to a task, domain, or tenant.

Purpose. Base LLMs are general-purpose.

Adaptation. Fine-tuning adapts vocabulary, style, judgment.

Concrete example (finance)

Fine-tune RoBERTa and Nemotron 3 Nano 4B on FOMC sentences to classify tone as **hawkish, dovish, or neutral**.

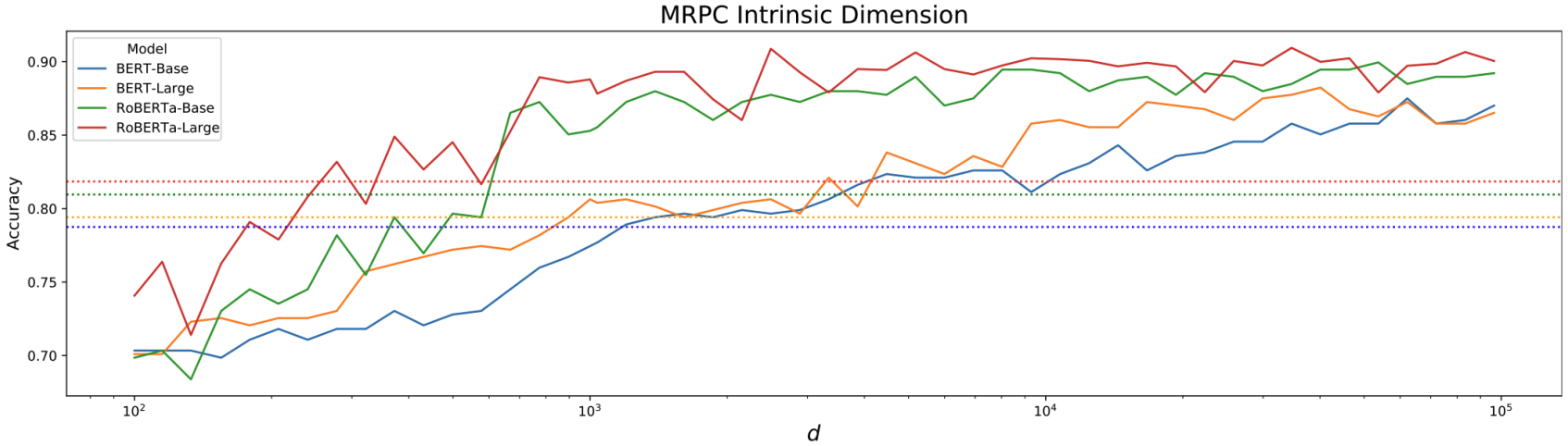


Fine-tuning = base model + custom dataset -> specialized model

Figure: AI Bites, YouTube.

Pretrained models already know most of what you need

Pretrained language models live in a much lower-dimensional subspace than their parameter count suggests.



MRPC accuracy for BERT / RoBERTa vs. intrinsic dimension d. Near-full accuracy at d far below the parameter count.

Aghajanyan, Gupta, Zettlemoyer 2020

What matrix rank means

Rank = number of independent directions in a matrix. Low-rank ⇒ redundancy ⇒ can be stored smaller.

Concrete example: 3 × 3 matrix

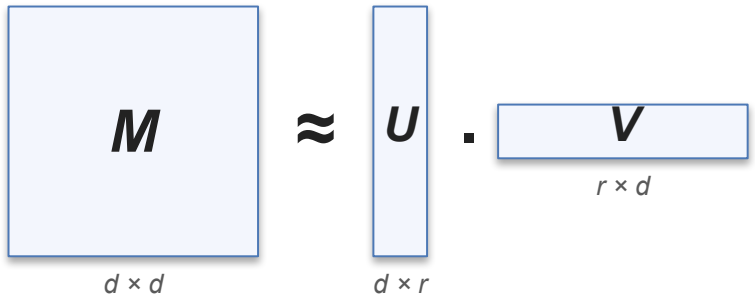
$$M = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 1 & 2 \end{bmatrix}$$

col 1 *col 2* *col 3*

Observation: col 3 = col 1 + col 2.
Only 2 independent columns.

rank(M) = 2

Low-rank factorization



d = 4096, r = 8

(4096 × 4096) → (4096 × 8) + (8 × 4096)

16.7M → 65K, 256× smaller

Low-**R**ank **A**daptation (LoRA)

LoRA: constrain the update to be low-rank

Same training objective as full fine-tuning, but only two small matrices change.

Objective (same as full fine-tuning).

$\max_{\Theta} \Sigma \log P_{\Phi_0+\Delta\Phi(\Theta)}(y_t \mid x, y_{<t})$

with $|\Theta| \ll |\Phi_0|$ *(typically 0.01 to 1%).*

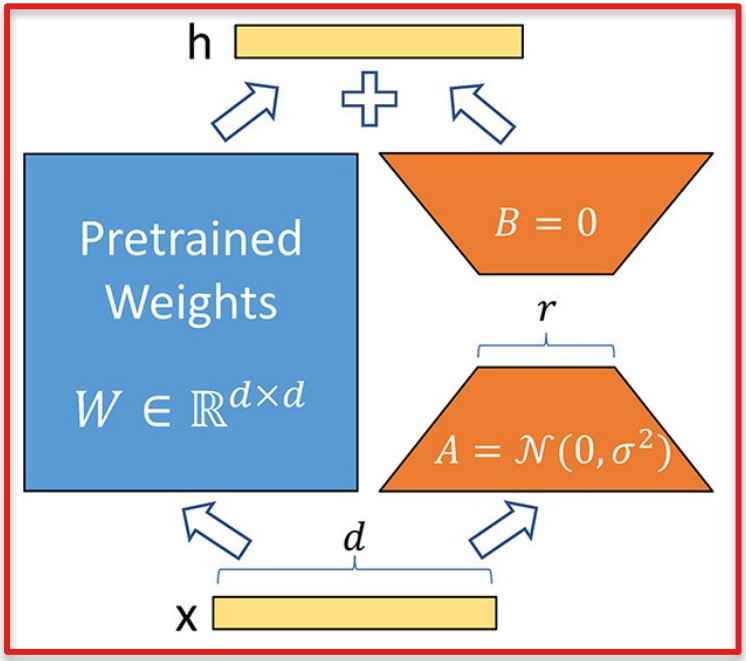
Hypothesis.

If pretrained weights already live in a low-rank subspace, then the updates ΔW during fine-tuning should be low-rank too.

Parameterization.

Freeze W_0. Learn a low-rank **ΔW = B · A** with rank $r \ll d$.

Trainable: B and A only. Frozen: W_0.



Hu et al. 2021 (eq. 1-2).

Train B and A separately. Merge them into W at serving time.

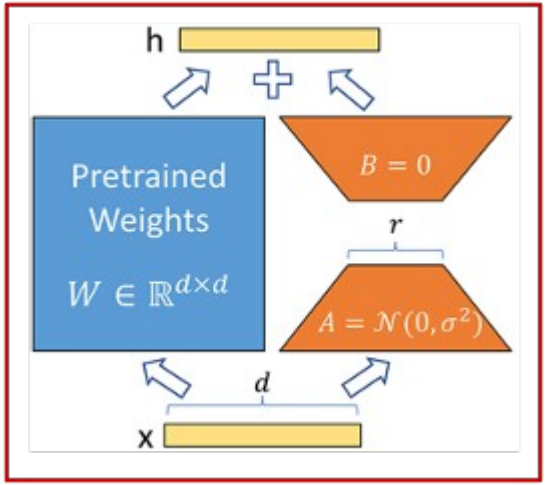
During training

Pretrained weights $W_0 \in \mathbb{R}^{d \times k}$ are frozen.

$h = W_0 x + \Delta W x = W_0 x + B A x$

$B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$

Two forward paths: the frozen one and the low-rank one.



Reparametrization: only A and B are trained.

During inference

Pre-merge: $W = W_0 + B A$. Store one matrix, run it once.

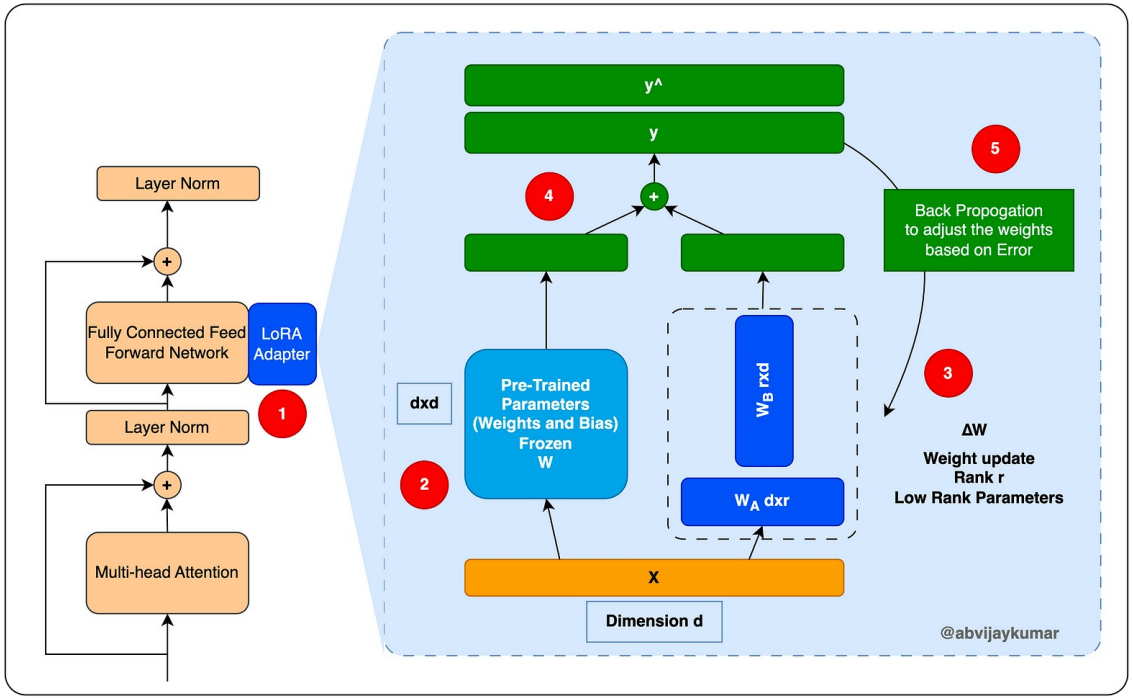
$h = W x$

No extra parameters at inference. Zero latency overhead.

Hu et al. 2021

A LoRA layer, step by step

1. Add a LoRA adapter beside the linear layer.
2. Freeze the pretrained W(d×d).
3. Add trainable $\Delta W = W_B W_A$, rank $r \ll d$.
4. Forward: $y = Wx + \Delta Wx$.
5. Backprop updates only W_A and W_B.



Which weights, and how large should r be?

Attention projections are usually enough. r = 8 or r = 16 is a safe default.

Which projection weights to adapt?

· Just W_q → works

· $\{W_q, W_v\}$ → covers most tasks

· All-linear (Q, K, V, O + MLP) → closes the remaining gap on hard tasks

· Modern default (HF PEFT): *target_modules='all-linear'*

What rank r?

· r = 1 → already competitive on many tasks

· r = 8–16 → standard for instruction-tuning

· r = 32–64 → used for harder capability shifts

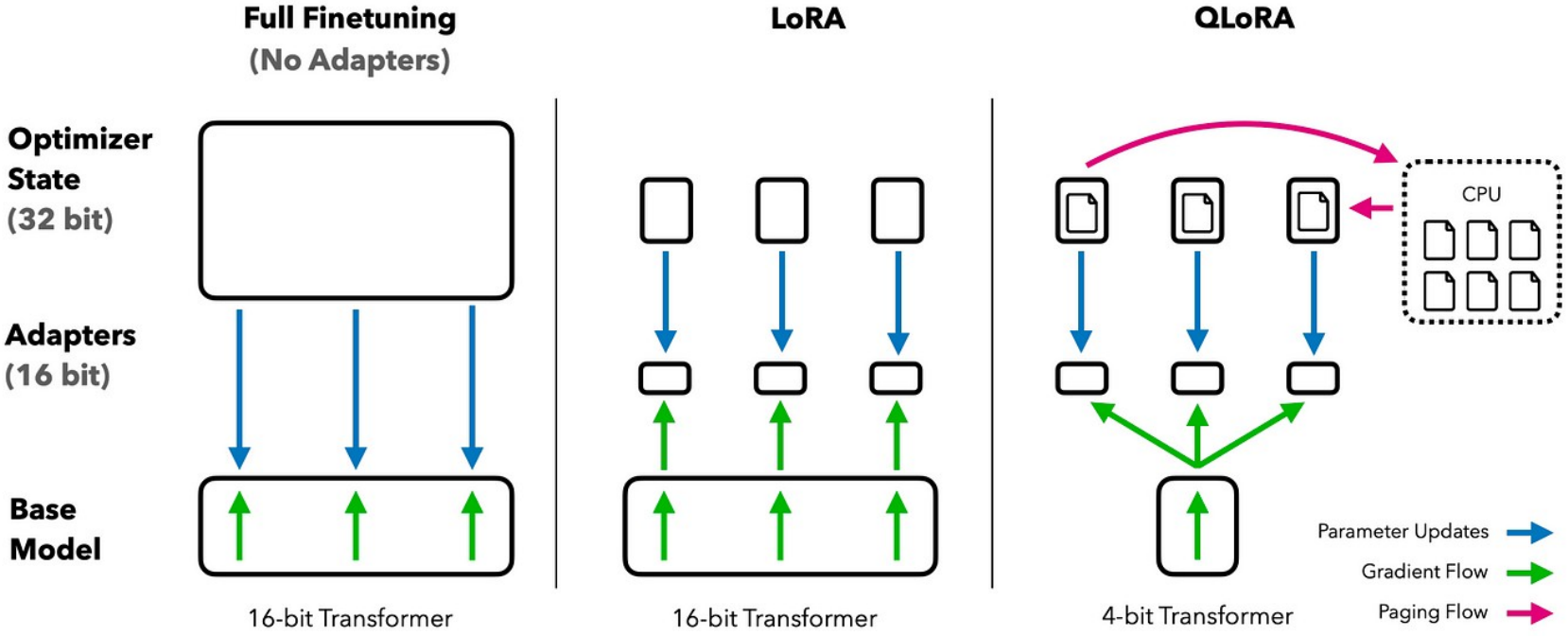
· Concrete example: W_q = 100 × 100 (10,000 params) vs B · A at r = 3 = 600 params → ≈17× reduction

Hu et al. 2021

Quantized bases: less memory for the frozen weights

LoRA shrinks the training cost, not the base weights

On Nemotron 4B, LoRA pulls peak training memory from ~38 GB to 9 GB. QLoRA pushes it further to 5.4 GB by quantizing the base.



Dettmers et al. 2023

The QLoRA recipe: four ingredients

Each attacks a specific memory cost. On this laptop (Nemotron 4B), 38 GB -> 5.4 GB with ~0.01 F1 given up.

1. NF4 quantization	4-bit weights. Data type designed for the normal distribution that pretrained weights actually follow.
2. Double Quantization	Quantize the quantization constants too. Saves another ~0.4 bits per parameter (~3 GB on a 65B model).
3. Paged Optimizers	NVIDIA unified memory + CPU spillover: absorbs memory spikes from long input sequences without OOM.
4. LoRA on top	Frozen quantized base + trainable low-rank adapters in 16-bit. All the LoRA benefits apply.

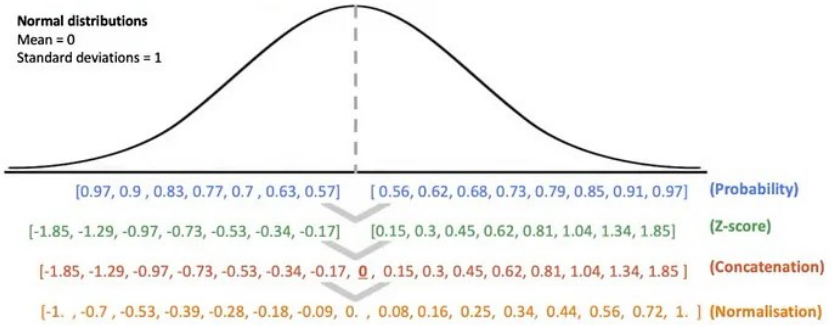
Dettmers, Pagnoni, Holtzman, Zettlemoyer 2023

NF4: 4-bit tuned for how weights are actually distributed

Why quantile bins beat any other 16 numbers.

The idea. Equal-probability quantile bins under $\mathcal{N}(0, 1)$.

The shape. Dense near 0, sparse in tails.



Quantile bins: dense near 0 where weights actually live, sparse in the tails.

Dettmers et al. 2023

NF4 vs naive Int4 and FP4

Where each format puts its 16 bins. NF4 aligns bins with where weights actually live.

Int4



Uniform spacing. Half the bins land in tails where weights rarely go.

FP4



Denser in mid-range (magnitude scale). Closer, but still not matched to a normal.

NF4



Bins land where weights live. Equal-probability quantiles under $\mathcal{N}(0, 1)$.

Empirically: NF4 > FP4 > Int4 on perplexity across every model size.

Dettmers et al. 2023

Two more memory tricks in QLoRA

Two smaller wins that keep single-GPU fine-tuning stable through memory spikes.

Double Quantization

Problem. 32-bit constants per block = 0.5 bits/param on 65B.

Fix. Quantize the constants too: 8-bit, blocked by 256.

Result. ~0.37 bits/param saved ≈ ~3 GB on 65B.

Paged Optimizers

Problem. Long sequences cause transient memory spikes, then OOM.

Fix. Unified memory spills optimizer state to CPU RAM during spikes.

Result. No batch-size cut, no user code change.

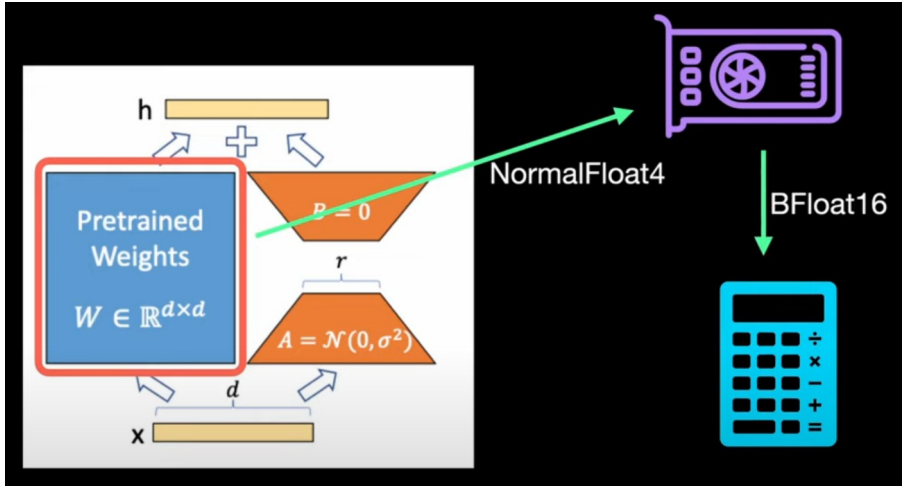
Dettmers et al. 2023

QLoRA is a training-memory tool, not an inference-latency tool

The cost. Every matmul dequantizes **NF4** to BF16, computes, then discards. Memory wins; compute suffers.

The fix. Blackwell runs **NVFP4** matmul natively: 4-bit in, 4-bit out, no dequant round-trip.

Effect. QLoRA-at-inference has hardware built for its data type.



Using PEFT

How teams use PEFT in practice

Task- or tenant-specific adapters

One base model. Small **LoRA adapters per task, domain, or customer**, updated without touching the base.

Multi-LoRA serving with vLLM

One base-model deployment shared across many adapters. The right adapter is **chosen per request**, not per model.

PEFT across the post-training stack

The same LoRA mechanism backs both supervised fine-tuning and preference optimization (**DPO**).

What a real fine-tune looks like today

Hugging Face PEFT + bitsandbytes: the QLoRA setup we ran on Nemotron 3 Nano 4B.

```
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForCausalLM, BitsAndBytesConfig

# 4-bit NF4 base. Plain LoRA: drop quantization_config, load in bf16.
bnb = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
                         bnb_4bit_use_double_quant=True,
                         bnb_4bit_compute_dtype="bfloat16",
                         llm_int8_skip_modules=["lm_head", "out_proj"])
base = AutoModelForCausalLM.from_pretrained(
    "nvidia/NVIDIA-Nemotron-3-Nano-4B-BF16", quantization_config=bnb)

model = get_peft_model(base, LoraConfig(r=64, lora_alpha=128,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                    "up_proj", "down_proj", "in_proj"]))
```

Worked example: hawkish or dovish?

Classify Fed (FOMC) sentences by policy tone. Data, splits, and metric from Shah et al. 2023.

The task

Label each sentence **hawkish** (toward raising rates), **dovish** (toward cutting), or **neutral**. ~2,400 labeled sentences, scored by weighted F1.

Three models

RoBERTa-base (125M) and **RoBERTa-large (355M)** from the paper, plus **Nemotron 3 Nano 4B**, an LLM scored on the label words.

33 runs

No fine-tuning, full fine-tuning, LoRA, QLoRA, three seeds each. Nemotron full fine-tuning does not fit in 24 GB (slide 3).

One laptop GPU

Dell Pro Max 16 Plus with an **NVIDIA RTX PRO 5000 Blackwell** laptop GPU (24 GB).

Task and data: Shah et al. 2023 (Combined-S split, seeds 5768, 78516, 944601).

Did the fine-tune actually help?

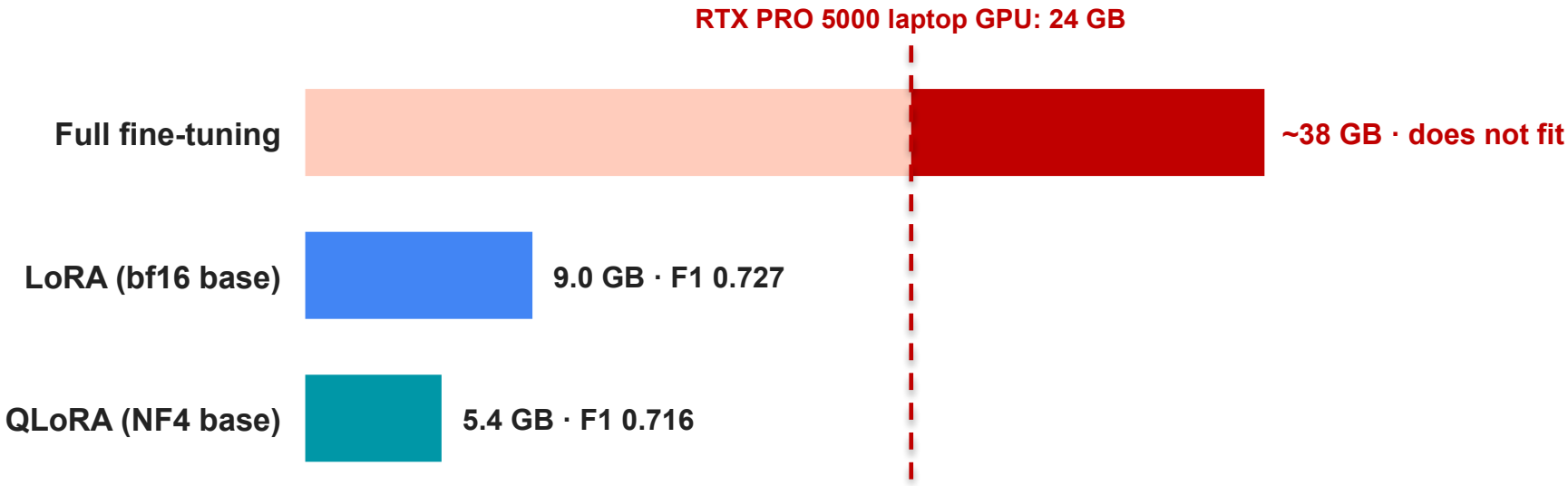
Test weighted F1 (0 to 1, higher is better), mean of three seeds.

Model	No fine-tuning	Full FT	LoRA	QLoRA
RoBERTa-base (125M)	0.188	0.667	0.684	0.685
RoBERTa-large (355M)	0.173	0.716	0.720	0.705
Nemotron 3 Nano 4B	0.622	*does not fit*	**0.727**	0.716

All runs ours, RTX PRO 5000 Blackwell laptop GPU. Data and splits: Shah et al. 2023 (Combined-S).

The QLoRA tradeoff on one laptop GPU

Peak training memory, Nemotron 3 Nano 4B. QLoRA gives up ~0.01 F1 for 40% less memory.



Plain LoRA already fits on a laptop GPU. QLoRA buys headroom for a bigger model or longer sequences.

Measured peak GPU memory during training (3-seed runs). Full fine-tuning: bf16 estimate from slide 3.

Flyte: 33 runs as one pipeline

One workflow runs unchanged on a MacBook, a laptop GPU, and a Flyte cluster.

Cheap reruns

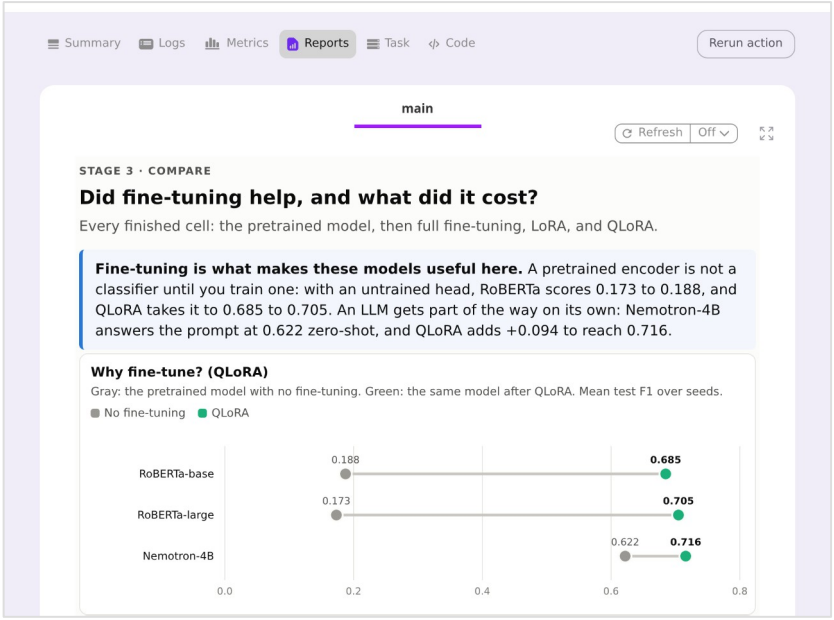
Data prep is **cached**. A failed GPU task **retries from its last epoch**, not from scratch.

Every stage explains itself

Each step writes a **report**: label mix, training curves, the model's most confident mistakes.

Results side by side

The compare stage puts every model and method on one page (**right**).



Flyte Reports tab: the compare stage, QLoRA vs no fine-tuning.

Serving the adapter with vLLM

The adapter trained on the laptop works unchanged on NVIDIA's FP8 release of the base model.

ACCURACY HOLDS	WEIGHTS IN GPU MEMORY	THROUGHPUT
0.725	**5.2 GB**	**~100**
weighted F1 (0.727 in training)	FP8 base + LoRA adapter	sentences per second

- **Flyte's vLLM plugin** deploys the model as a service from the same project.
- **OpenAI-compatible API**: existing clients work unchanged.
- **Multi-LoRA**: several adapters share one base; the adapter is picked per request.

vLLM on an RTX PRO 5000 Blackwell laptop GPU, 496 test sentences, 3 seeds. Base: nvidia/NVIDIA-Nemotron-3-Nano-4B-FP8. vLLM

Appendix

Anticipated questions, depth material, precision + memory accounting.

Q: How do you pick the rank r in practice?

What is the optimal rank for the factorized matrices? I was wondering to know if you have findings on choosing the optimal rank in the experiment.

	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI ($\pm 0.1\%$)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

Table 6: Validation accuracy on WikiSQL and MultiNLI with different rank r. To our surprise, a rank as small as one suffices for adapting both W_q and W_v on these datasets while training W_q alone needs a larger r. We conduct a similar experiment on GPT-2 in Section H.2.

Q: The VRAM savings claim looks too good. Where does it come from?

What does VRAM stand for? If it is GPU memory, then the 2/3 saving of training memory is quite surprising to me:LoRA still needs to build the forward and backward computation graphs for the whole model. Could the authors clarify?

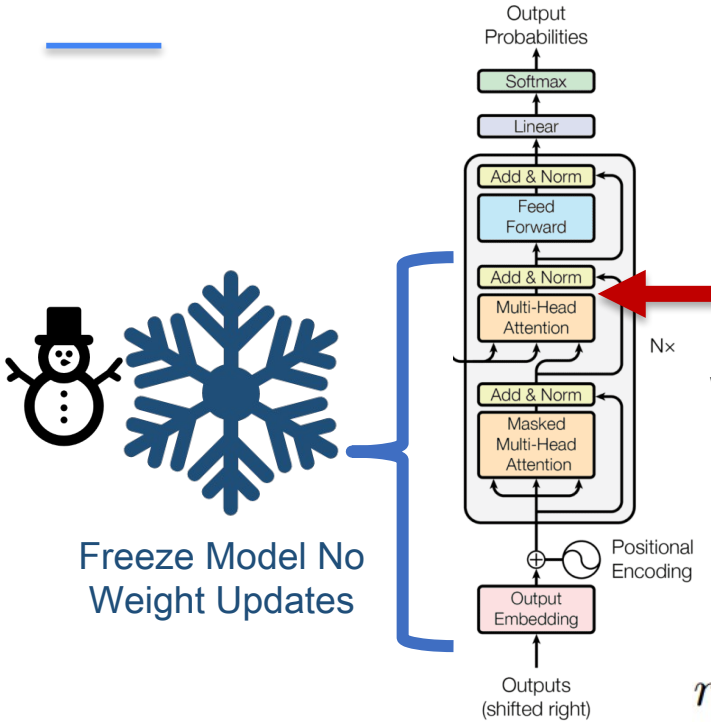
- “VRAM” refers to “Video RAM” or “GPU memory.”
- We do not need to store the adaptive optimizer states (e.g., momentum and second order moment for Adam) for the pretrained parameters
- For smaller models where VRAM is dominated by activations, the savings will be less

Q: Does NF4 generalize to other bit-widths?

Can the structure of NF4 data type be directly plugged into 8-bit and higher precision data-types for improved performance, if yes is there any experiment to show the prowess of the NFX data types?

- NormalFloats can easily be defined for any bit-width.
- NF3 > FP3, but not stable

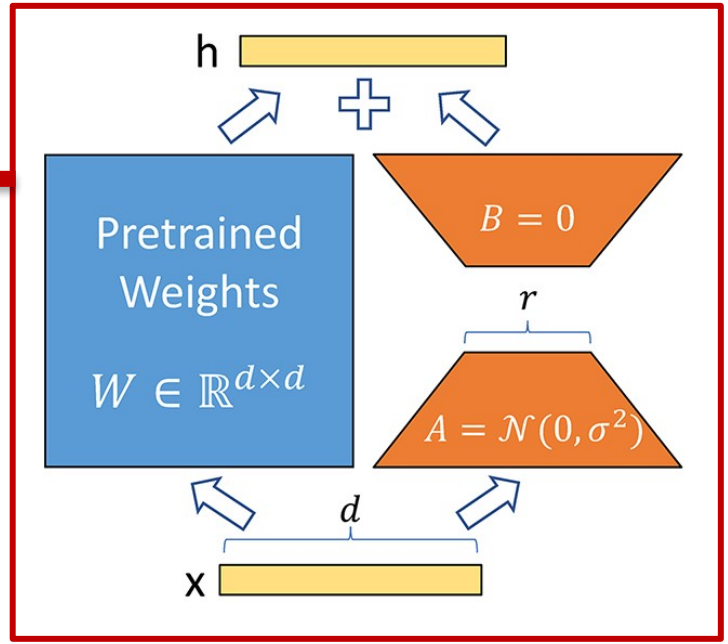
LoRA: freeze the model, train two small matrices



Freeze Model No Weight Updates

Only Update Weights of LoRA Matrices

$$B \in \mathbb{R}^{d \times r}$$
$$A \in \mathbb{R}^{r \times k}$$
$$r \ll \min(d, k)$$



$$h = W_0 x + \Delta W x = W_0 x + BAx$$

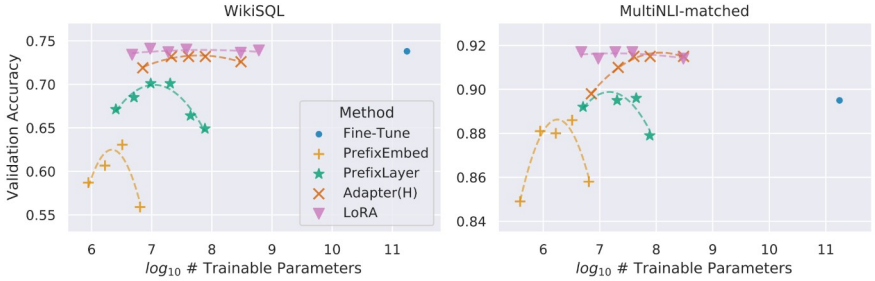
Hu et al. 2021

Evidence: LoRA matches full FT on GPT-3 175B

~4.7M trainable params reaching or exceeding full 175B fine-tune performance.

Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	**73.8**	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 ($\text{Adapter}^{\text{H}}$)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 ($\text{Adapter}^{\text{H}}$)	40.1M	73.2	**91.5**	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	**91.7**	**53.8/29.8/45.9**
GPT-3 (LoRA)	37.7M	**74.0**	**91.6**	53.4/29.2/45.1

LoRA matches or exceeds full FT across WikiSQL, MNLI, SAMSum.



Scalability: LoRA vs prefix-tuning and adapters, GPT-3.

Hu et al. 2021

Evidence: 4-bit QLoRA matches 16-bit full fine-tuning

MMLU 5-shot across LLaMA 7B–65B. 4-bit NF4 with DQ ≈ BFloat16 full FT.

	Mean 5-shot MMLU Accuracy								
LLaMA Size	7B		13B		33B		65B		Mean
Dataset	Alpaca	FLAN v2	Alpaca	FLAN v2	Alpaca	FLAN v2	Alpaca	FLAN v2	
BFloat16	38.4	45.6	47.2	50.6	57.7	60.5	61.8	62.5	53.0
Float4	37.2	44.0	47.3	50.0	55.9	58.5	61.3	63.3	52.2
NFloat4 + DQ	39.0	44.5	47.5	50.7	57.3	59.2	61.8	63.9	53.1

65B model. 48 GB single GPU. No accuracy loss vs 16-bit full fine-tune.

Dettmers et al. 2023

Prompt-based PEFT: prefix tuning, P-tuning v2

· Prompt engineering: hand-crafted natural-language instructions. Zero training. Brittle.
· Prefix tuning (Li & Liang '21): learn a small set of continuous 'prefix' vectors prepended to inputs.
· P-Tuning v2: learnable prompts at every layer, not just the input. Closer to full FT in capacity.
· Downside vs LoRA: eats input-token budget; usually plateaus below LoRA on hard tasks.
· Upside: can be composed with LoRA (prefix + LoRA together: increasingly common).

Li & Liang 2021

Modern quantization neighbors: GPTQ, AWQ, EXL2, PolarQuant, NVFP4

The quantization stack around QLoRA in 2026.

· GPTQ (Frantar '22): post-training weight quantization for INFERENCE. Different tool from QLoRA.
· AWQ (Lin '23): activation-aware weight quantization: protects the ~1% of weights that matter most for activations. Standard for serving.
· EXL2 (2024): variable-bit-per-weight quantization tuned per-model. Popular in llama.cpp world.
· PolarQuant (2025): rotates weight distributions before quantization: better for outliers than NF4.
· TurboQuant (2025): hardware-aware quantization for specific accelerator layouts.
· NVFP4 (NVIDIA, 2024): 4-bit float (E2M1) with per-block scaling, native to Blackwell tensor cores. NF4's hardware-accelerated cousin: designed for serving, not training.
· How QLoRA fits: QLoRA quantizes the base for TRAINING. GPTQ / AWQ / EXL2 / NVFP4 quantize a trained model for SERVING. Different pipelines, same math family.

LoRA under post-training: SFT, DPO, RFT stack on top

PEFT and post-training are orthogonal axes, they stack.

· SFT (Supervised Fine-Tuning): the default target for QLoRA. Cross-entropy on instruction-response pairs. 10 lines of code shown earlier.

· DPO (Direct Preference Optimization): train on (chosen, rejected) preference pairs. Skips the RLHF reward model. Works over a LoRA-adapted base cleanly.

· RFT (Reinforcement Fine-Tuning, GRPO/RLOO/etc.): online RL from rewards. Usually done with LoRA to keep the RL loop cheap. Only trainable params get updated.

· Typical production pipeline: base → QLoRA SFT for domain → DPO on top for preference alignment → merge or serve unmerged.

· For legal-review workloads: DPO is where reviewer-accepted-vs-rejected redlines become the alignment signal.

DPO (Rafailov et al. 2023)

Attention variants (MLA, GQA, FA3) and target_modules

Attention architecture changes what "LoRA the attention" even means.

· GQA (Grouped-Query Attention): fewer K, V heads than Q heads. Fewer projections → fewer LoRA target modules to think about.

· MLA (Multi-head Latent Attention, DeepSeek): compresses K and V into a shared low-rank latent. Now "LoRA the K projection" doesn't mean the same thing: there IS no separate K projection.

· FlashAttention-3: kernel-level fusion. Doesn't change PEFT semantics but changes the memory profile you're optimizing against.

· Practical PEFT implication: for modern architectures (Llama-3, DeepSeek-V3, Qwen-2.5), target_modules='all-linear' is the safe default because you can't easily reason about individual attention projections anymore.

GQA (Ainslie et al. 2023)

LongLoRA: extending context length under LoRA

Fine-tune a short-context base into a long-context model with a LoRA-shaped budget.

· Problem: attention memory scales $O(n^2)$ with context length. Fine-tuning a 4K-context base to 32K context in full FT is prohibitive.

· LongLoRA: LoRA-adapt attention + shifted-sparse attention (S^2-Attn) → fine-tune 7B to 100K context on a single 8×A100 node.

· S^2-Attn: compute attention within groups + overlap groups by shifting. Approximate but effective during training; merged back at inference.

· Why relevant here: legal documents are LONG. If domain adaptation includes a context-length extension, LongLoRA is the standard tool.

Chen et al. 2023

The broader PEFT taxonomy: AdaLoRA, IA³, BitFit

LoRA-family is not the only PEFT-family.

· AdaLoRA (Zhang '23): allocate rank adaptively per weight matrix instead of using a single r everywhere.

· IA³ (Liu '22): learn per-channel scaling vectors instead of low-rank matrices: extremely few params, less capacity.

· BitFit (Zaken '22): fine-tune ONLY the bias terms. Nearly-zero param count, surprisingly competitive baseline.

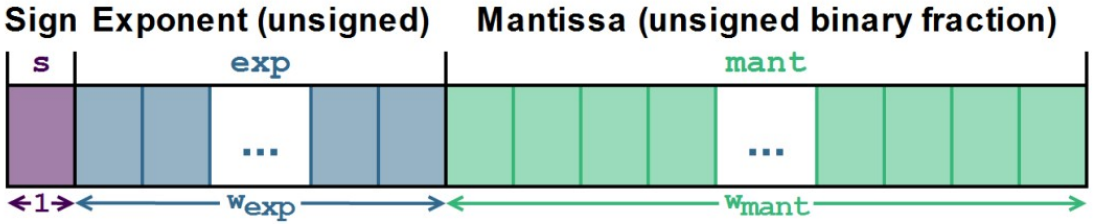
· (IA)³ and BitFit are the honest "how few params can you get away with?" experiments.

· AdaLoRA is the honest "do you need the same rank everywhere?" experiment (answer: no).

· When to reach for these: extreme storage/serving-count constraints (thousands of tenants), or as sanity-check baselines against LoRA.

AdaLoRA (Zhang et al. 2023)

Floating Point (IEEE 754): sign, exponent, mantissa



Format width: $w_f = 1 + w_{exp} + w_{mant}$

Exponent bias: $bias = 2^{(w_{exp}-1)} - 1$

Encoded normal value: $v = (-1)^s \times 2^{(exp-bias)} \times 1.mant$

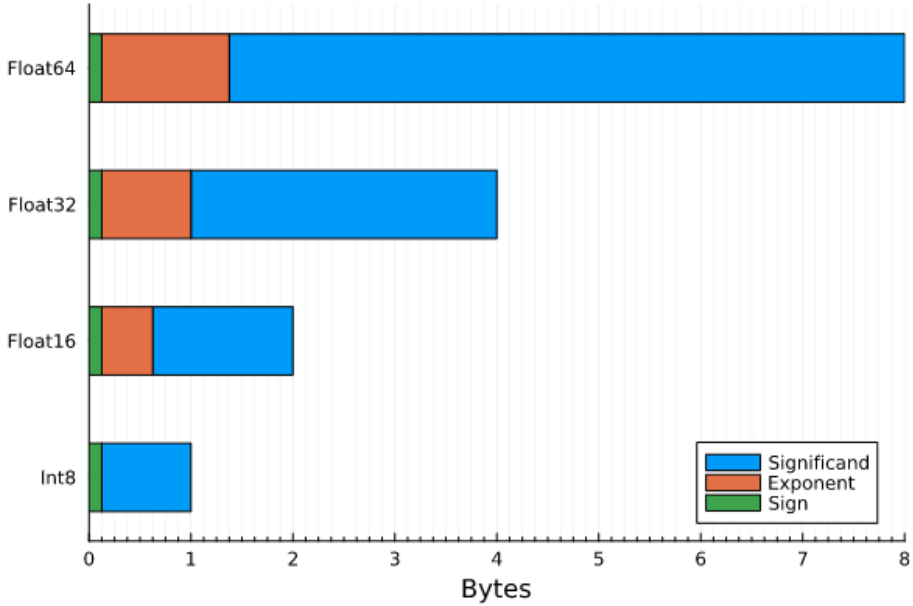
Special cases:

$$exp = 0 \Rightarrow v = (-1)^s \times 2^{(1-bias)} \times 0.mant$$

$$exp = 2^{w_{exp}} - 1 \wedge mant = 0 \Rightarrow v = (-1)^s \times \infty$$

$$exp = 2^{w_{exp}} - 1 \wedge mant \neq 0 \Rightarrow v = NaN$$

Mixed-Precision Floating Point: fp64 vs fp32 vs fp16 vs int8



blog.eleuther.ai/transformer-math/

Why AdamW is so expensive: momentum + variance = 2x weights

AdamW is an **Optimizer**.

Per weight, it stores momentum (m) and variance (v).

7B weights → 14B extra numbers = the **Optimizer** bucket.

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla w_t$$

$$v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * (\nabla w_t)^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} * \hat{m}_t$$

Loshchilov & Hutter 2019 (Decoupled Weight Decay Regularization, ICLR 2019).

Large Language Model on 16 GB VRAM: full QLoRA accounting

